

Hibernate Annotation Config Implementation — Walkthrough

SME explanation of the sample Hibernate implementation referenced from [TutorialsPoint](#) (Hibernate Annotations). Covers Object/Relational mapping via JPA/Hibernate annotations directly inside the persistence class, and the supporting hibernate.cfg.xml configuration.

Topics Covered

- How Object-to-Relational mapping is done directly in the persistence class (Employee.java)
- @Entity, @Table, @Id, @GeneratedValue, @Column annotations
- Hibernate Configuration (hibernate.cfg.xml) — Dialect, Driver, Connection URL, Username, Password

1. Object-to-Relational Mapping in the Persistence Class (Employee.java)

With the annotation-based approach, Hibernate **removes the need for a separate .hbm.xml mapping file**. Instead, the mapping metadata is written directly inside the Java class itself, using annotations from the **javax.persistence** (JPA) package. This keeps the class definition and its table mapping together in one place, which makes the code easier to read and maintain.

```
import javax.persistence.*;

@Entity
@Table(name = "EMPLOYEE")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "salary")
    private int salary;

    public Employee() {}

    // getters and setters
    public int getId() { return id; }
```

```

public void setId(int id) { this.id = id; }
public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }
public String getLastName() { return lastName; }
public void setLastName(String lastName) { this.lastName = lastName; }
public int getSalary() { return salary; }
public void setSalary(int salary) { this.salary = salary; }
}

```

Compare this to the XML approach from Hands-on 2: there, **Employee.hbm.xml** declared the class → table and field → column mapping externally. Here, the exact same information is expressed with annotations placed right above the class declaration and each field — the class itself *is* the mapping definition. Hibernate reads these annotations at startup (instead of parsing a .hbm.xml file) to build the same internal metadata model it would have built from XML.

2. Core Annotations Explained

@Entity

```

@Entity
public class Employee { ... }

```

- Marks a POJO class as a **persistent entity** — telling Hibernate/JPA that this class should be mapped to a database table.
- Without @Entity, Hibernate has no way of knowing that this class needs to be tracked and persisted; it's the annotation equivalent of the top-level **<class>** tag in the old .hbm.xml file.
- Every entity class must have a no-argument constructor (as required by JPA) and a field annotated with @Id.

@Table

```

@Table(name = "EMPLOYEE")

```

- Specifies the exact **database table** this entity maps to.
- **Optional** — if omitted, Hibernate assumes the table name is the same as the class name. It's used explicitly whenever the table name differs from the class name (as here: class **Employee**, table **EMPLOYEE**).
- Can also specify a **schema** or **catalog**, and unique constraints across columns, using additional attributes.

@Id

```

@Id
private int id;

```

- Marks the field as the entity's **primary key**.
- Every entity must declare exactly one (or a composite) @Id field — this is what Hibernate uses to uniquely identify each row/object, and what methods like **session.get()** key off of.

@GeneratedValue

```

@GeneratedValue(strategy = GenerationType.IDENTITY)

```

- Tells Hibernate **how the primary key value should be generated** automatically, instead of the application having to set it manually.
- Common strategies: **IDENTITY** (relies on an auto-increment column in the database), **SEQUENCE** (uses a database sequence object), **TABLE** (uses a separate table to simulate a sequence), and **AUTO** (lets Hibernate pick the best strategy for the underlying database).
- This is the annotation equivalent of the `<generator class="...">` element nested inside `<id>` in the XML mapping file from Hands-on 2.

@Column

```
@Column(name = "first_name")
private String firstName;
```

- Maps a specific field to a specific **database column**.
- **Optional** when the field name and column name already match — Hibernate infers it implicitly. Required (or at least recommended) when they differ, as with `firstName` → `first_name`.
- Also supports attributes like **length** (size of a String column), **nullable** (whether the column allows NULLs), and **unique** — giving fine control over the generated schema, similar to what the `<property>` tag offered in the XML mapping file.

3. Hibernate Configuration (hibernate.cfg.xml)

Even with annotation-based mapping, Hibernate still needs a place to configure the **database connection** and **global settings** — this doesn't move into the annotations. It stays in **hibernate.cfg.xml**, exactly as in the XML mapping approach, except the `<mapping>` entry now points at an annotated class instead of an `.hbm.xml` file.

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">
      jdbc:mysql://localhost/test</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">pass123</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

    <mapping class="com.example.Employee"/>
  </session-factory>
</hibernate-configuration>
```

Dialect

- **hibernate.dialect** tells Hibernate which SQL flavor to generate for the target database (e.g. **MySQLDialect**, **OracleDialect**, **PostgreSQLDialect**).
- Since different databases have slightly different SQL syntax and data types, the dialect lets Hibernate produce SQL that's valid for the specific database in use, without the application code needing to change.

Driver

- **hibernate.connection.driver_class** specifies the fully-qualified JDBC driver class for the database (e.g. **com.mysql.cj.jdbc.Driver** for MySQL).

- This is the same driver class you'd normally load when connecting to that database directly via plain JDBC — Hibernate uses it internally to open connections.

Connection URL

- **hibernate.connection.url** is the JDBC connection string identifying the database server, port, and schema/database name to connect to (e.g. **jdbc:mysql://localhost/test**).

Username

- **hibernate.connection.username** is the database login username Hibernate uses when opening connections.

Password

- **hibernate.connection.password** is the corresponding database password for that username.
- In real projects these credentials are usually externalized (environment variables, a secrets manager, or a properties file excluded from version control) rather than hard-coded in `hibernate.cfg.xml`.

4. How This Gets Used — SessionFactory Setup

The annotated class plugs into the same Hibernate API you saw in Hands-on 2 — only the mapping source changes (annotations instead of `.hbm.xml`). Building the `SessionFactory` and performing CRUD operations (`session.save()`, `session.get()`, `session.createQuery().list()`, `session.delete()`, `commit()/rollback()`) works identically once the `SessionFactory` is built:

```
SessionFactory factory = new Configuration()
    .configure() // reads hibernate.cfg.xml
    .addAnnotatedClass(Employee.class)
    .buildSessionFactory();

Session session = factory.openSession();
Transaction tx = null;

try {
    tx = session.beginTransaction();

    Employee emp = new Employee();
    emp.setFirstName("Zara");
    emp.setLastName("Ali");
    emp.setSalary(5000);

    session.save(emp);

    tx.commit();
} catch (Exception ex) {
    if (tx != null) tx.rollback();
    ex.printStackTrace();
} finally {
    session.close();
}
```

Note the one structural difference: **`.addAnnotatedClass(Employee.class)`** (or a `<mapping class="...">` entry in `hibernate.cfg.xml`) registers the annotated class with the `SessionFactory`, replacing the `<mapping resource="Employee.hbm.xml">` line used in the XML-mapping approach. Everything downstream — `Session`, `Transaction`, `save/get/query/delete` — behaves exactly the same either way.

XML Mapping vs. Annotation Mapping — Quick Comparison

Aspect	XML Mapping (Hands-on 2)	Annotation Mapping (Hands-on 3)
Where mapping lives	Separate Employee.hbm.xml file	Inside Employee.java itself
Class → table	<class name=... table=.../>	@Entity + @Table(name=...)
Primary key	<id> + <generator>	@Id + @GeneratedValue
Field → column	<property column=.../>	@Column(name=...)
Registered via	<mapping resource="...hbm.xml"/>	<mapping class="..."/> or addAnnotatedClass()

Reference: *TutorialsPoint — Hibernate Annotations* (tutorialspoint.com/hibernate/hibernate_annotations.htm)